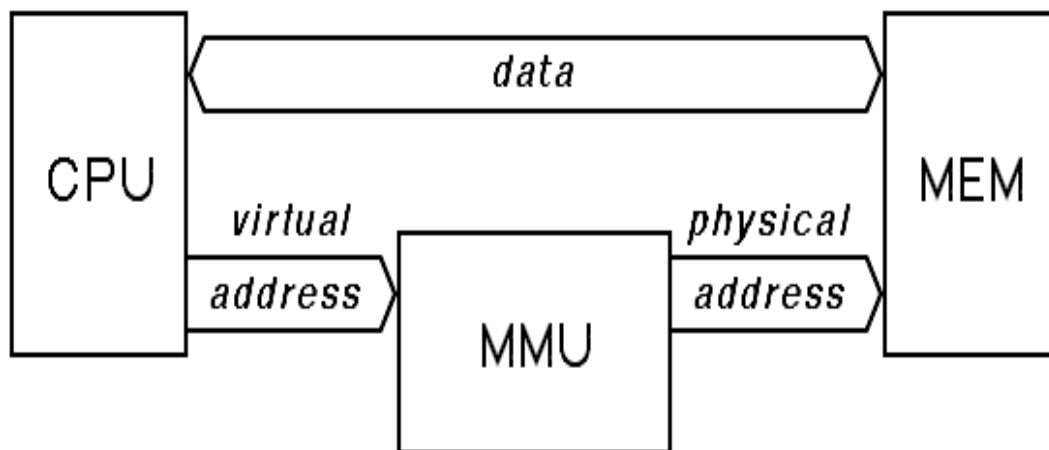


Unit-3

Memory management unit: -

- A memory management unit (MMU), sometimes called paged memory management unit (PMMU), is a computer hardware component responsible for handling accesses to memory requested by the CPU.
- Its functions include translation of virtual addresses to physical addresses (i.e., virtual memory management), memory protection, cache control, bus arbitration and in simpler computer architectures.
- The MMU normally translates virtual page numbers to physical page numbers via an associative cache called a translation look-aside buffer (TLB).
- Hardware device that maps logical/virtual to physical address.
- In MMU the value in the relocation register is added to every address generated by a program at the time the address is sent to memory.
- The program deals with logical addresses; it never sees the real physical addresses.



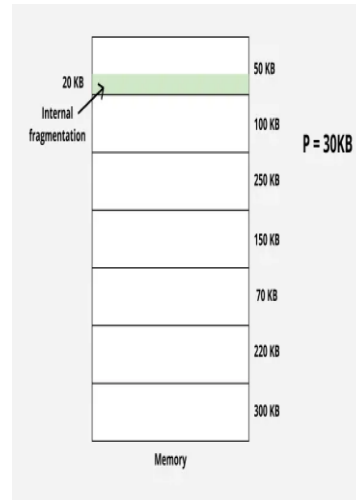
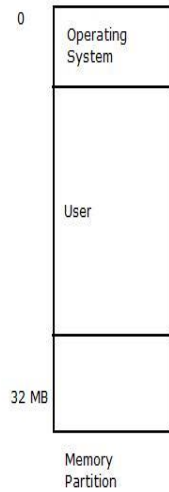
Memory Management Schemes: -

Memory allocation: -

- Memory is a central part of the modern computer systems.
- Memory is large collection storage cells. The size of each storage cell is 1 byte with its own memory address.
- The CPU fetches the instructions from memory according to the value of program counter.
- After the instruction has been executed, result may be stored back in memory.
- There are two Commands of memory allocation. [1] Contiguous memory allocation. [2] Non contiguous memory allocation.

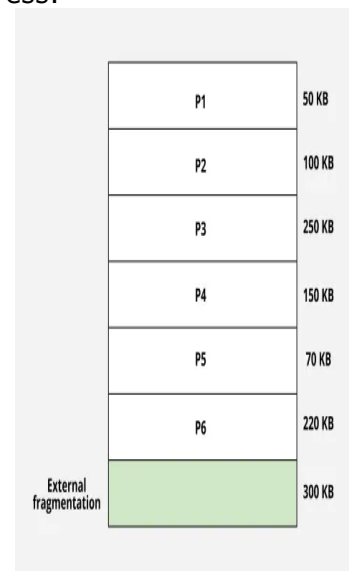
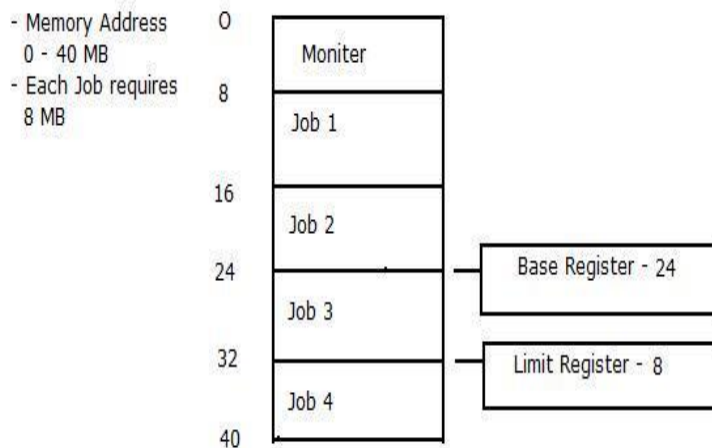
Contiguous memory allocation: -

- The main memory must allocate to both, the operating system and the various user process.
- The memory is usually divided into two parts. One for the operating system and another for user process.
- It is possible to place the operating system in either low memory or higher memory.
- It is more common to place the operating system in low memory. The user process can take place in high memory.

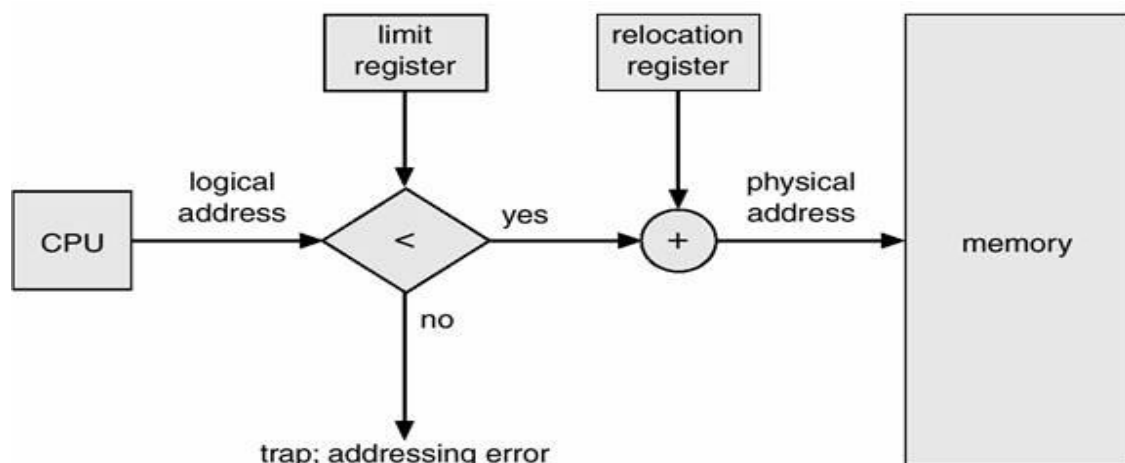


Single partition allocation:

- If the operating system is loaded into low memory and the user processes are executing in high memory, we need to protect the operating system code and data from changes by the user process.
- We also need to protect the user processes from one another.
- We provide this protection by using a relocation register.
- Consider the following figure.
- The base register is also known as relocation register.
- The relocation register contain the value of physical address.



- The limit register contains the range of logical address.
- By using relocation and limit register the memory management unit provides the memory to the particular job.
- Relocation-register scheme used to protect user processes from each other, and from changing operating-system code and data.



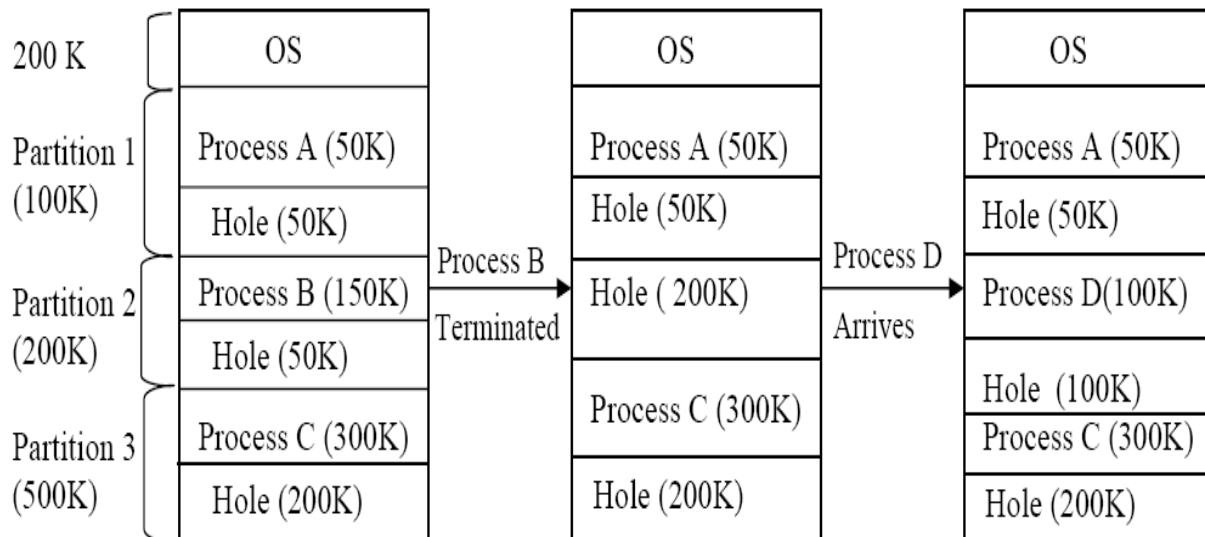
Non-Contiguous memory allocation: -

- A process can be spanned across different spaces in the main memory in a non-consecutive manner. Suppose process P of size 4KB. Consider main memory has two empty slots each of size 2KB. Hence total free space is, $2 \times 2 = 4$ KB. In contiguous memory allocation
- **Physical address space:** Main memory (physical memory) is divided into blocks of the same size called frames. frame size is defined by the operating system by comparing it with the size of the process.
- **Logical Address space:** Logical memory is divided into blocks of the same size called process pages. page size is defined by hardware system and these pages are stored in the main memory during the process in non-contiguous frames.

Multiple partition allocation:

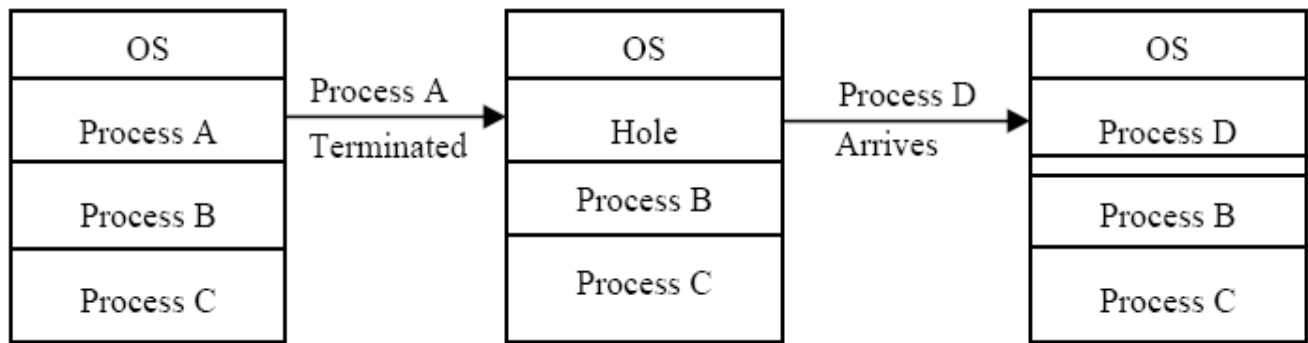
Fixed partition:

- There are several processes are stored into memory at the same time.
- One of the simplest schemes for the memory allocation is to divide the memory into a number of fixed sized partitions.
- Each partition contains only one process at a time.
- When the partition is free, a process is selected from the input queue and allocated to that free partition.
- When the process is completed, that partition becomes available for another process.
- The operating system keeps a table indicating which part of memory are available and which part of memory is allocated.



Dynamic (Variable) partition:

- Initially, all memory is available for the user process, considered as one large block of available memory.
- That large block is known as a **hole**.
- When a process arrives and needs memory, we search for a **hole** large enough for this process and allocate only as much required.
- **First-fit:** Allocate the first hole that is big enough.
- **Best-fit:** Allocate the smallest hole that is big enough.
- **Worst-fit:** Allocate the largest hole.

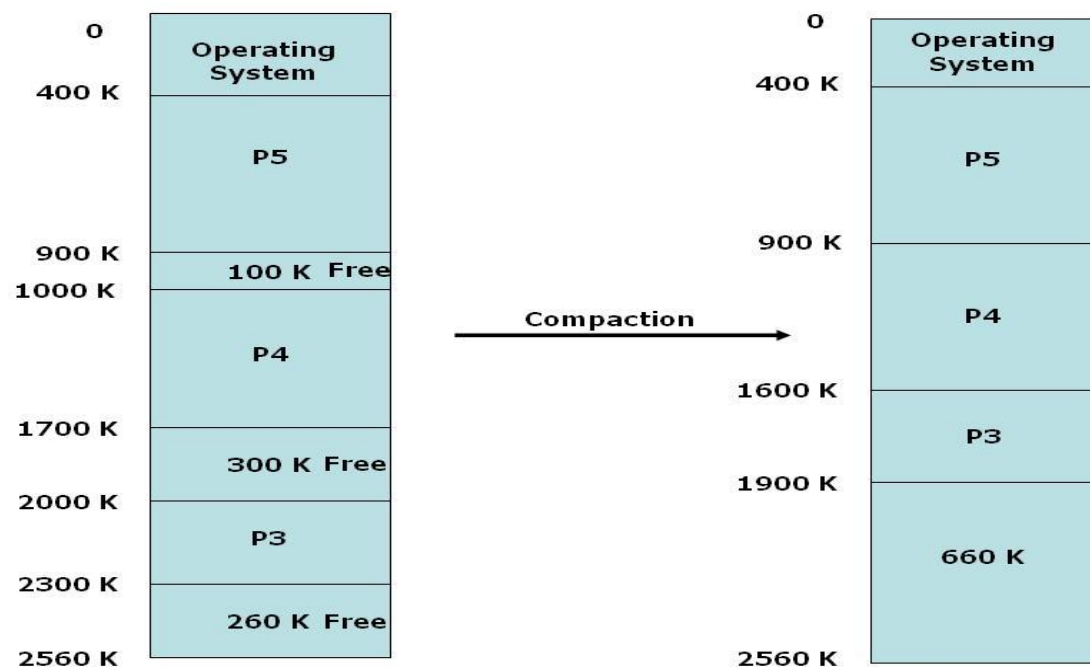


Fragmentation:

- Process is stored and removed from memory which create free memory space which are too small to use by other processes.
- The process not able to allocate to memory blocks because its small size and memory blocks always remain used is called fragmentation.
- This type of problem happens during a dynamic memory allocation system where free blocks are quite small so it is not able to fulfil any request.

External and Internal Fragmentation:

- Multiple partition allocation schemes suffer from external fragmentation.
- As processes are loaded and removed from the memory, the free memory space is broken into little pieces.
- External fragmentation exists when enough total memory space exists to satisfy a request, but it is not contiguous; storage is fragmented into a number of holes.
- Internal fragmentation is memory that is internal to a partition, but is not being used.
- One solution to the problem of external fragmentation is **compaction**.
- The compaction is used to reshuffle the all free memory places together and create large block.
- **External Fragmentation** – Total memory space exists to satisfy a request, but it is not contiguous.
- **Internal Fragmentation** – Allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.



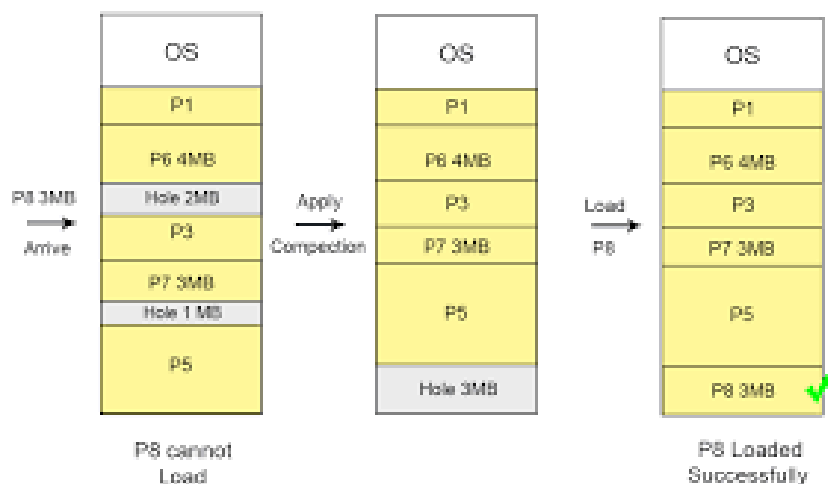
- Reduce external fragmentation by compaction
 1. Shuffle memory contents to place all free memory together in one large block.
 2. Compaction is possible only if address binding is dynamic, and is done at execution time.

Defragmentation: -

- Defragmentation is the process of rearranging the data on a storage medium such as a hard drive.
- Defragmentation hard drive can improve a computer or laptop performance and speed reduce fragmentation.
- Defragmentation is a process used in operating system to optimize the performance an organization of computer hard drive or storage media.
- Fragmentation or scattered across different location on the disk this fragmentation can slow down the read and write process as the system has to search and access multiple area to file.
- Defragmentation can provide several benefits. It improves system performance by reducing the time it takes retrieve data from the hard disk.
- Defragmentation works by rearranging the fragmentation file and organization them in contiguous locations.
- The system can access file more efficiently resetting in faster read and write operations.
- Defragmentation is the process of reorganizing the data on a disk so that files are stored in **contiguous (continuous) blocks**, rather than being scattered in fragments across the disk. This improves the speed of data access and the overall performance of the system.
- When files are created, modified, or deleted over time, **free spaces** develop in various parts of the disk.
- New data may not fit into a single large block, so it gets stored in **multiple non-contiguous blocks**. This is called **fragmentation**.

Compaction: -

- **Compaction** is the process of **shifting memory contents** in RAM to **combine all free memory space into a single contiguous block**. It is used to solve the problem of **external fragmentation** in memory management.
- In **dynamic memory allocation**, processes are loaded and removed from memory over time.
- This creates **holes** (unused spaces) between occupied memory blocks.
- Even if total free memory is enough, a large process might **not fit** because free space is **scattered**.
- **Compaction rearranges memory** so that all free space is **collected together**.

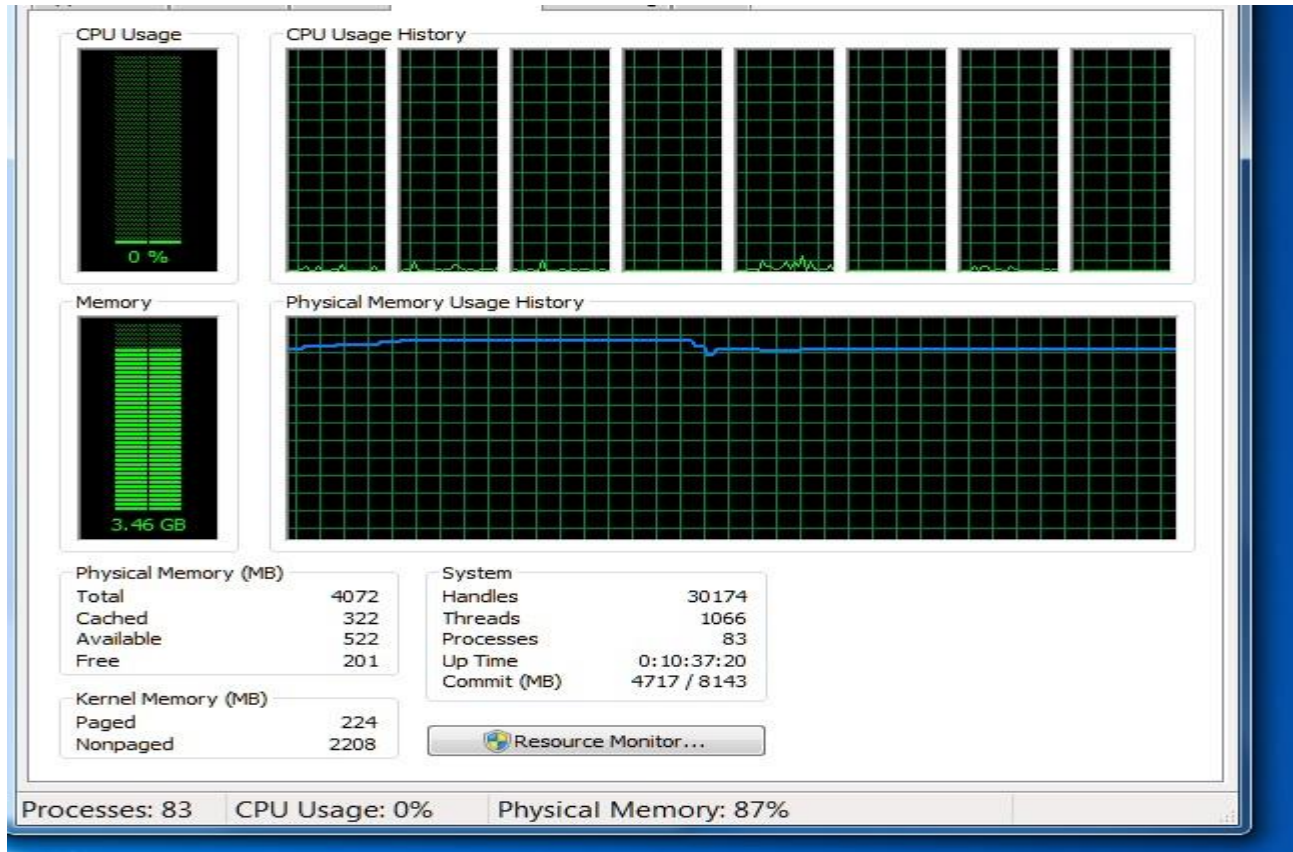


Virtual memory: -

- Virtual memory is a common part of most operating systems on desktop computers.
- Most computers today have something like 256 or 512 megabytes of RAM available for the CPU to use.
- Unfortunately, that amount of RAM is not enough to run all of the programs that most users expect to run at once.
- For example, if you load the operating system, an e-mail program, a Web browser and word processor into RAM simultaneously, 256 megabytes is not enough to hold it all.
- If there were no such thing as virtual memory, then once you filled up the available RAM your computer would have to say, "Sorry, you can not load any more applications. Please close another application to load a new one."
- With virtual memory, computer is look at RAM for areas that have not been used recently, that data are copied onto the hard disk. This frees up space in RAM to load the new application.
- The area of the hard disk that stores the RAM image is called a page file.

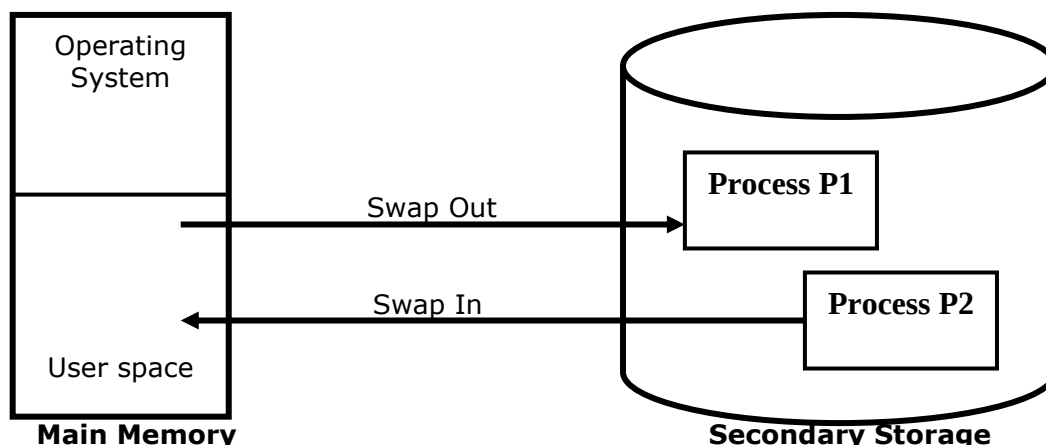
Physical memory: -

- The physical memory of a computer usually consists of Random Access Memory (RAM) chips and hard drives.
- RAM is the amount of real storage, and is the total amount of memory installed on a computer.
- Physical memory also referred to as the physical storage or the real storage.
- Physical memory is a term used to describe the total amount of memory installed in the computer.
- For example, if the computer has two 64MB memory modules installed, it has a total of 128MB of physical memory.



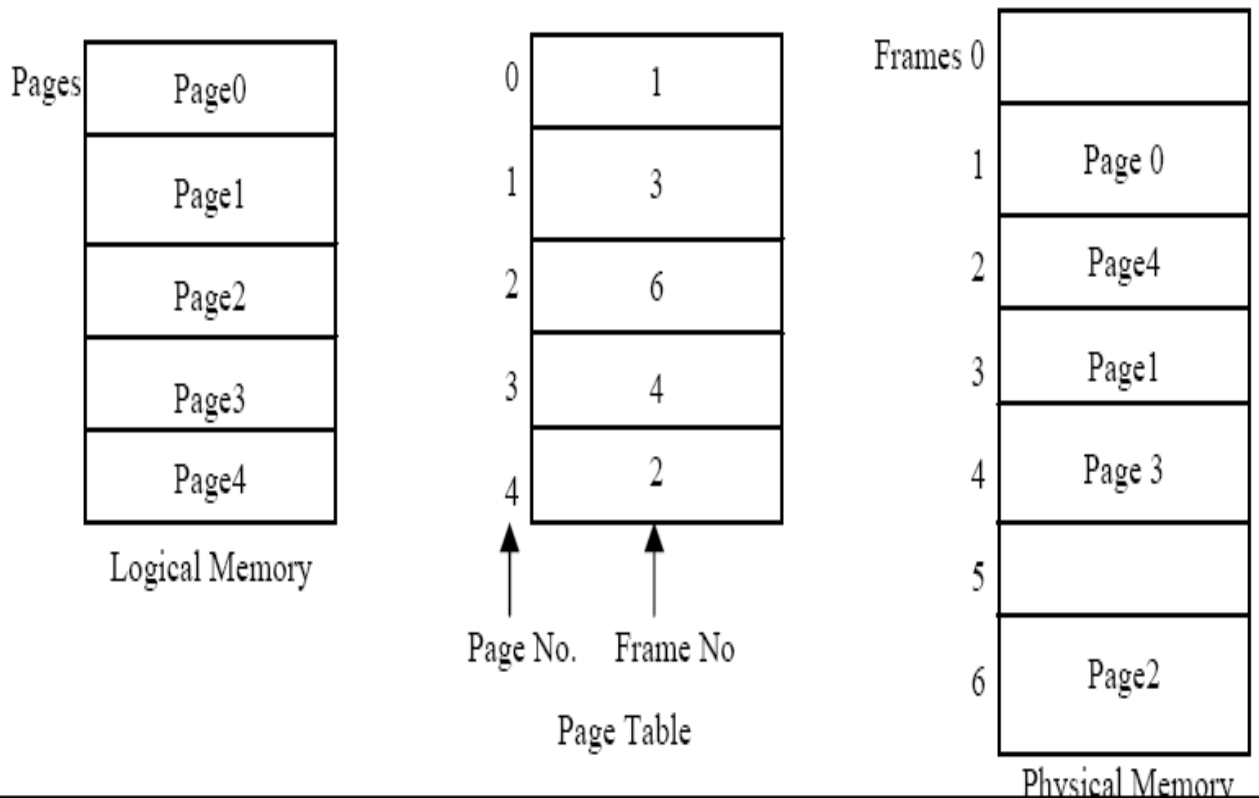
Swapping: -

- A process can be swapped out temporarily from memory and stored in to secondary storage, and then back into memory for continued execution.
- This swapping policy is used for priority-based scheduling.
- If a higher priority process arrives in memory and wants service, the memory manger can swap out the lower priority process in secondary storage.
- The higher priority process can load into memory and execute that process.
- When the higher priority process finished, the lower priority process can be swapped back from secondary storage into memory and continued the execution.
- This process is also known as roll-out, roll-in process.



Paging:

- Physical memory is broken into fixed sized blocks called frames.
- Logical memory is broken into blocks of the same size called pages.
- Keep track of all free frames.
- To run a program of size n pages, need to find n free frames and load program.
- The reason behind this is implementation of paging mechanism using page number and page offset.
- Set up a page table to translate logical to physical addresses.
- Remove/reduce external fragmentation. Internal fragmentation exists.



- Page table: used to translate logical to physical addresses.
- **Address translation scheme: -**
 - Address generated by CPU is divided into:
 - Page number (p)
 - Used as an index into the page table.
 - Page table contains base address of each page in physical memory.
 - page offset (d)
 - Combined with base address to define the physical memory address sent to the memory unit.
 - Use page number to index the page table.
 - Get the page frame start address.
 - Add offset with that to get the actual physical memory address.
 - Access the memory.

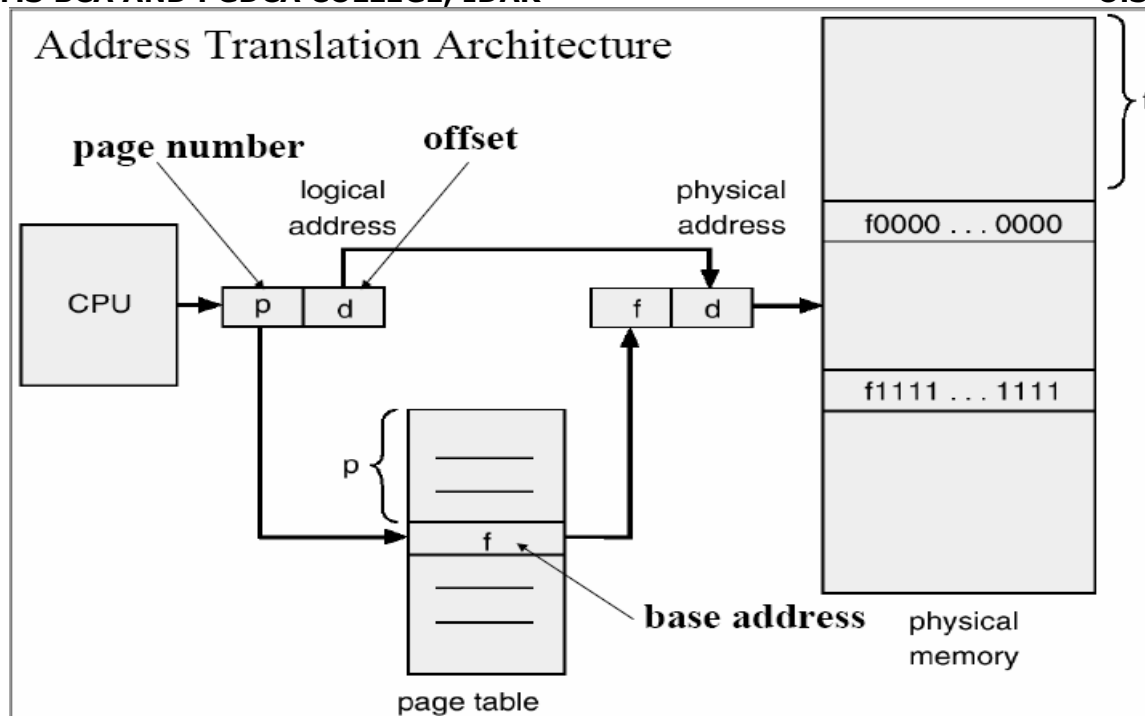


Figure: - Address Translation Architecture

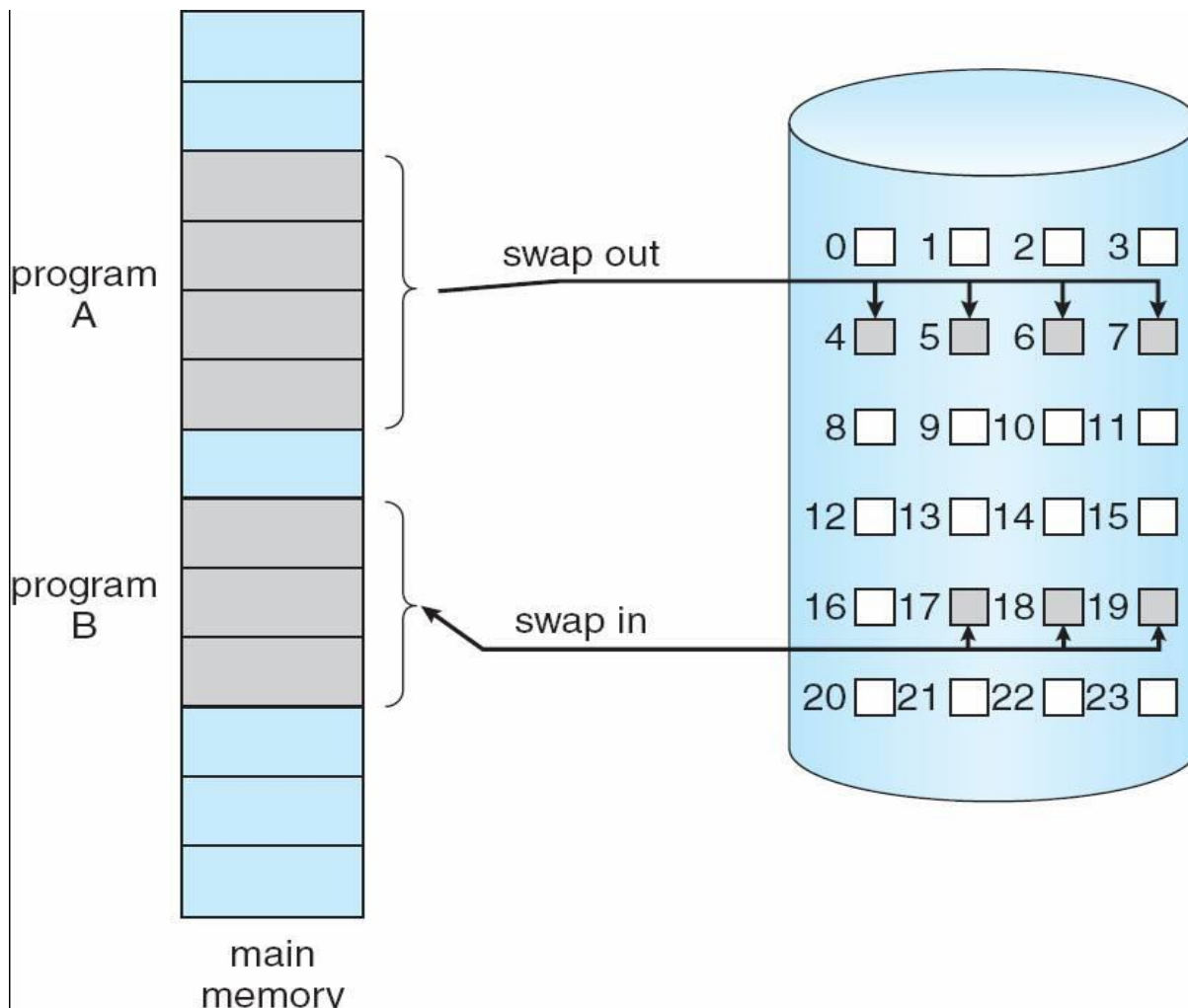


Figure: - Example of Paging

Demand paging: -

- In operating systems, demand paging is a method of virtual memory management.
- Consider how an executable program might be loaded from disk into memory.
- One option is to load the entire program in physical memory at program execution time.
- However, a problem with this approach is that we may not initially need the entire program in memory.
- An alternative strategy is to initially load pages only as they are needed.
- This technique is known as demand paging and is commonly used in virtual memory systems.
- A demand-paging system is similar to a paging system with swapping where processes reside in secondary memory (usually a disk).
- When we want to execute a process, we swap it into memory. Rather than swapping the entire process into memory, however, we use a lazy swapper.
- A lazy swapper never swaps a page into memory unless that page will be needed.

- A swapper manipulates entire processes, whereas a pager is concerned with the individual pages of a process. We thus use pager, rather than swapper, in connection with demand paging.



Page fault: -

- A page is a fixed-length block of memory that is used as a unit of transfer between physical memory and external storage like a disk, and a page fault is an interrupt (or exception) to the software raised by the hardware, when a program accesses a page that is mapped in address space, but not loaded in physical memory.
- If a Page-table mapping indicates an absence of the page in physical memory, hardware raises a "Page- Fault".
- OS traps this fault and the interrupt handler services the fault by initiating a disk-read request.
- Once page is brought in from disk to main memory, page-table entry is updated and the process which faulted is restarted.
- May involve replacing another page and invalidating the corresponding page-table entry.
- An invalid page fault or page fault error occurs when the operating system cannot find the data in virtual memory.
- This usually happens when the virtual memory area, or the table that maps virtual addresses to real addresses, becomes corrupt.

I/O Management: -

- I/O (Input/Output) management in an operating system refers to the OS's responsibility for controlling and managing all input and output operations between the computer and its peripherals (like keyboards, screens, printers, etc.).
- It involves techniques to handle data transfer, device access, and error handling, ensuring efficient and coordinated interaction between hardware and software.
- The operating system in computer I/O is manage and control I/O operations and I/O devices. I/O service provided by operating system and these service in the application I/O interface.

Program Controlled I/O: -

- There is no permission through which I/O can inform to CPU about data transfer.

- In this technique, the **CPU is in direct control** of all I/O operations. The processor continuously checks (polls) the status of the I/O device until it is ready to send or receive data.
- I/O sets its own status or waits. CPU runs program periodically and checks the status of each device one by one.
- If any device has its status set CPU performs data transfer for it. It works on the principle of polling.
- The processor issues I/O command on behalf of a process to an I/O module that process then busy wait for the operation to be completed before processing.

Interrupt Driven I/O: -

- **Interrupt Driven I/O** is a technique where the **CPU does not continuously check (poll)** the I/O device. Instead, the CPU continues executing other instructions, and the I/O device **sends an interrupt signal** when it is ready for data transfer.
- I/O device has a provision (interrupt signal) to inform to CPU about communication.
- When CPU receives interrupt its complete execution of current instruction is saved. The status is stored to correct process the stored.
- CPU issues a command to the I/O device. Instead of waiting, CPU continues executing other instructions. When the device is ready, it sends an **interrupt signal** to the CPU. The CPU stops its current execution (context switch), saves its state, and runs the **Interrupt Service Routine (ISR)**.
- The interrupt handler routine is at a fixed address in memory. Most CPUs have two interrupt request lines. One is the **nonmaskable** interrupt which is reserved for events such as unrecoverable memory error.
- **Maskable:** it can be turned off by the CPU before the execution of critical instruction sequence that must not be interrupted. The maskable interrupt is used by device controllers to request service.
- The interrupt mechanism accepts an address or a number that selects a specific interrupt handling routine from a small set. In most architectures this address is an offset in a table called the **interrupt vector**.